Maîtrise des Structures Répétitives, Preuve de Terminaison et Invariants de Boucle

Thème principal : Structures de contrôle itératives et preuves formelles d'algorithmes (Terminaison et Correction)

Synthèse exécutive : Ce cours pose le socle algorithmique indispensable de la programmation structurée : l'itération. Répéter des instructions sans dupliquer le code source élimine l'anti-pattern du 'copier-coller' et permet de traiter des jeux de données volumineux ou dynamiques. Le cours établit une grille décisionnelle rigoureuse entre boucles bornées (POUR) et non bornées (TANT QUE, RÉPÉTER... JUSQU'À), tout en introduisant les deux piliers formels de la vérification logicielle : le variant de boucle, qui garantit la terminaison (absence de boucle infinie), et l'invariant de boucle, qui démontre par récurrence la correction mathématique du résultat produit.

Typologie des Structures Itératives et Grille de Décision Algorithmique

En programmation structurée, une structure de contrôle itérative permet d'exécuter un bloc d'instructions de manière répétée sans recopier textuellement ce code. L'informaticien doit choisir la structure exacte selon deux questions hiérarchiques : 1) Connaît-on le nombre exact d'itérations à l'avance ? Si oui, la boucle 'POUR' s'impose : elle encapsule l'initialisation du compteur, le test de borne et l'incrémentation automatique, éliminant tout risque d'oubli d'évolution. 2) Si le nombre de tours dépend d'un événement dynamique (saisie utilisateur, lecture d'un flux, seuil de convergence numérique), on choisit une boucle non bornée. Si le traitement peut être ignoré dès le départ, on utilise le 'TANT QUE' (0 à N itérations). Si le traitement doit impérativement s'exécuter au moins une première fois (cas classique du menu interactif ou du contrôle de saisie avec message d'erreur), on choisit le 'RÉPÉTER... JUSQU'À' (1 à N itérations).

- Anti-pattern du Copier-Coller : dupliquer manuellement des blocs d'instructions produit un code rigide, inmaintenable et incapable de manipuler des volumes de données dynamiques.
- Itération Bornée (POUR) : structure réservée aux cas où le nombre d'itérations est connu avant d'entrer dans la boucle. La variable compteur est gérée automatiquement par la structure.
- Itération Non Bornée à Pré-condition (TANT QUE) : la condition de continuation est évaluée avant chaque passage. Si la condition est fausse à l'initialisation, le corps s'exécute 0 fois.
- Itération Non Bornée à Post-condition (RÉPÉTER... JUSQU'À) : le corps s'exécute d'abord, puis la condition d'arrêt est testée. Elle garantit au minimum 1 exécution.

Cas concret enseignant : *L'analogie de la piste d'athlétisme : si l'entraîneur ordonne 'Fais 10 tours', c'est une boucle POUR car la borne est connue d'avance. S'il ordonne 'Cours tant qu'il ne pleut pas', c'est une boucle TANT QUE : s'il pleut déjà au vestiaire, le coureur ne foule même pas la piste (0 tour). S'il ordonne 'Fais un tour et arrête-toi dès que tu es essoufflé', c'est une boucle RÉPÉTER... JUSQU'À : le coureur effectue obligatoirement son premier tour avant d'évaluer son niveau de fatigue.*

Preuve Formelle de Terminaison : Le Variant de Boucle

Pour prouver de façon absolue qu'un algorithme ne restera pas bloqué indéfiniment dans une boucle non bornée, l'étudiant doit exhiber un 'variant de boucle'. Un variant est une grandeur scalaire entière V dépendant de l'état des variables. La démonstration formelle exige de prouver deux axiomes : 1) La décroissance stricte : pour toute itération k, $V_{k+1} < V_k$. 2) La minoration : il existe une valeur plancher M telle que si $V <= M$, la condition de boucle est violée. En vertu de la propriété fondamentale des entiers naturels (ensemble bien ordonné), une suite d'entiers strictement décroissante et minorée est obligatoirement finie. Cette formulation garantit mathématiquement que la boucle terminera en un nombre fini d'itérations.

- Boucle Infinie : blocage critique du programme où la condition d'arrêt n'est jamais atteinte, monopolisant le processeur.
- Variant de Boucle (V) : fonction des variables de boucle à valeurs entières positives qui mesure la distance jusqu'à l'arrêt.

• Décroissance Stricte : le variant doit diminuer d'au moins 1 unité entière à chaque tour de boucle (V_{k+1} <= V_k - 1).

• Minorant / Borne Plancher : seuil entier (généralement 0) en dessous duquel le variant ne peut descendre sans que la condition de continuation devienne fausse.

• Principe du Bon Ordre : propriété mathématique établissant qu'il n'existe aucune suite d'entiers naturels strictement décroissante et infinie, ce qui garantit scientifiquement l'arrêt.

Cas concret enseignant : *La métaphore de la jauge d'essence d'un véhicule : à chaque kilomètre parcouru (une itération), le volume de carburant dans le réservoir (le variant V) diminue d'une quantité strictement positive. Puisque la quantité d'essence ne peut descendre sous 0 litre (le minorant), la voiture finira obligatoirement par s'immobiliser. Si la jauge ne baissait pas à chaque kilomètre, la voiture roulerait à l'infini : c'est l'illustration exacte de la boucle infinie.*

Preuve Formelle de Correction : L'Invariant de Boucle

Prouver qu'un programme termine ne suffit pas : il faut démontrer qu'il produit le résultat exact. L'invariant de boucle applique le principe du raisonnement par récurrence aux structures itératives. On identifie une propriété logique P liant les variables de calcul et les indices de parcours. On démontre que : 1) P est vérifié dès l'initialisation. 2) L'exécution du corps de boucle préserve la véracité de P (si P est vrai à l'état k, il est vrai à l'état k+1). 3) À la terminaison, la condition de continuation C est devenue fausse (non C) et P demeure vrai. L'expression logique (P et non C) permet de déduire formellement que l'accumulateur ou la variable finale détient la valeur spécifiée.

• Invariant de Boucle : prédicat logique (propriété P) qui reste rigoureusement VRAI avant la boucle, pendant chaque itération et à la sortie.

• Phase d'Initialisation : preuve que l'assertion P est vraie sur l'état initial des variables, avant la première itération (k=0).

• Phase de Conservation (Hérédité) : preuve que si P est vraie au début d'une itération k arbitraire, elle reste vraie à la fin de cette itération après exécution du corps.

• Phase de Conclusion (Correction) : conjonction logique de l'invariant final préservé (P) et de la négation de la condition de boucle (non C) établissant que la sortie est mathématiquement conforme au cahier des charges.

Cas concret enseignant : *L'analogie de la tournée du livreur de journaux : au départ de sa tournée matinale, l'égalité 'Journaux distribués + Journaux restants dans le sac = Total initial' est vraie. À chaque maison livrée, les deux nombres changent mais leur somme demeure rigoureusement invariante tout au long de la journée. À la fin, quand le sac est vide (condition de sortie), le nombre de journaux distribués est obligatoirement égal au total initial prévu.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Est-ce qu'on peut systématiquement remplacer une boucle POUR par une boucle TANT QUE ?
Réponse du professeur : Oui, d'un point de vue purement algorithmique, toute boucle POUR peut s'écrire sous la forme d'un TANT QUE en gérant manuellement l'initialisation, le test et l'incrémentation. Cependant, c'est une mauvaise pratique de conception quand le nombre d'itérations est connu d'avance : utiliser un TANT QUE dans ce cas expose au risque d'oublier l'incrémentation dans le corps (créant une boucle infinie) et surcharge inutilement le code.

Question étudiant : Si j'utilise une boucle TANT QUE avec la condition (i < N), est-ce que les instructions s'exécutent si i est déjà égal à N dès le départ ?
Réponse du professeur : Non, absolument pas. Dans une boucle TANT QUE, le test de la condition logique est effectué en pré-condition, c'est-à-dire avant même le tout premier tour. Si la condition est fausse dès l'entrée (ici si i est supérieur ou égal à N), le processeur saute directement après la structure sans exécuter le corps une seule fois (0 itération).

■ Exercice & Démonstration d'Application

Énoncé : Écrire un algorithme en pseudo-code permettant de calculer la somme des N premiers entiers pairs stricts (par exemple pour N=3, calculer la somme de 2 + 4 + 6 = 12). Justifier le choix de la boucle, fournir le pseudo-code complet et formaliser le variant de boucle garantissant la terminaison.

Résolution pas à pas :
Étape 1 : Justification du choix de la boucle. Le paramètre N est fourni en entrée du programme ; le nombre total d'itérations est donc parfaitement déterminé et borné à l'avance. La boucle 'POUR' est la structure de référence. Étape 2 : Définition des variables. - N : Entier (nombre d'éléments pairs à additionner) - i : Entier (compteur de boucle variant de 1 à N) - somme : Entier (accumulateur initialisé à 0) - terme : Entier (valeur du nombre pair courant, initialisé à 2) Étape 3 : Rédaction du pseudo-code formel. Algorithme SommePremiersPairs Variables : N, i, somme, terme : Entiers Début Lire(N) somme <- 0 terme <- 2 Pour i allant de 1 à N par pas de 1 Faire somme <- somme + terme terme <- terme + 2 FinPour Afficher(somme) Fin Étape 4 : Démonstration du variant de boucle. Si l'on exprime ce parcours sous la forme d'une boucle TANT QUE équivalente avec compteur (i initialisé à 1 et condition i <= N), l'expression entière V = (N - i + 1) constitue un variant de boucle valide : 1. À chaque passage, i est incrémenté de 1, donc V_{k+1} = N - (i + 1) + 1 = V_k - 1 (le variant décroît strictement de 1 à chaque tour). 2. La boucle continue tant que i <= N, ce qui équivaut à V >= 1. Dès que V atteint 0 (minorant), la condition de boucle devient fausse et l'algorithme s'arrête.

Résultat : Pour N = 3 : - Tour 1 (i=1) : somme = 0 + 2 = 2 ; terme devient 4. - Tour 2 (i=2) : somme = 2 + 4 = 6 ; terme devient 6. - Tour 3 (i=3) : somme = 6 + 6 = 12 ; terme devient 8. Sortie de boucle avec affichage de 12. La terminaison est formellement garantie par le variant V = (N - i + 1) et le résultat correspond exactement à la spécification.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la différence fondamentale à l'exécution entre une boucle TANT QUE et une boucle RÉPÉTER... JUSQU'À ?

R : La boucle TANT QUE évalue sa condition en pré-condition (avant le corps) et peut s'exécuter de 0 à N fois. La boucle RÉPÉTER... JUSQU'À évalue sa condition en post-condition (après le corps) et garantit donc au minimum 1 exécution (de 1 à N fois).

Q : Quelles sont les deux propriétés mathématiques strictes qu'une fonction entière doit vérifier pour constituer un variant de boucle valide ?

R : Elle doit être strictement décroissante à chaque itération (V_{k+1} < V_k) et minorée par une constante plancher (généralement 0) telle que l'atteinte de ce minorant rende la condition de boucle fausse.

Q : Quel oubli classique dans la rédaction d'une boucle TANT QUE entraîne systématiquement une boucle infinie et un zéro sur la question en DS de R1.01 ?

R : L'oubli de la mise à jour (incrémentation ou modification d'état) de la variable de contrôle dans le corps de la boucle, ce qui maintient la condition de continuation éternellement vraie.